

Arbiter

Árbitro de decisões técnicas

Documento de trabalho · Estado atual do projeto · Gerado em sessão de planejamento

Este documento é um retrato, não uma spec final. Várias decisões aqui ainda estão vivas e podem mudar. Em particular, o schema e a regra de conflito foram refinados ao longo do planejamento, e o teste de extração com LLM — a parte mais arriscada e que valida toda a premissa — ainda não foi executado. Trate como registro do estado atual do raciocínio, não como verdade fixa.

O que é

Um buscador de decisões reais de engenharia. Em vez de perguntar "Kafka ou RabbitMQ?" para um chatbot e receber uma resposta genérica, o usuário descreve **seu contexto** — "sistema de notificações, 50k usuários, time de 3 devs" — e o produto retorna o que empresas reais decidiram em situações parecidas, com a fonte original linkada.

O diferencial central é o **painel de conflito**: quando duas empresas chegaram a conclusões opostas sobre a mesma decisão, o produto mostra os dois lados explicitamente, com o contexto de cada uma. A resposta certa depende da situação — e é isso que o produto torna visível, em vez de esconder atrás de uma opinião única.

Por que este projeto

Demonstra exatamente o que as vagas de Full Stack, Back-end, Dados e Analytics Engineer pedem: pipeline de ingestão de dados, busca semântica com embeddings, backend com lógica de ranking, e frontend usável. Não é CRUD, não é dashboard, não é clone de produto existente.

O risco honesto: a qualidade do produto depende inteiramente da qualidade da extração dos dados. Se a ingestão produzir cards rasos ou com **reason** genérico, o painel de conflito — a única feature que diferencia o produto — fica vazio de significado mesmo cheio de cards. A extração do **reason** ancorado em contexto não é um detalhe do projeto: é o projeto.

Decisões de arquitetura já tomadas

Input

Consultas contextuais — o usuário descreve o problema com contexto (volume, tamanho do time, tolerância a complexidade), não apenas "X vs Y". Sem contexto, a busca retorna decisões de empresas em escalas irrelevantes para o caso.

Output

Cards de decisão com ranking por relevância ao contexto, e um painel de conflito que aparece **condicionalmente** — apenas quando existe divergência real entre duas decisões.

Fontes

ADRs públicos no GitHub + eng blogs de empresas (Stripe, Discord, Shopify, Uber, Netflix, Figma) + threads do Hacker News. **Critério de seleção confirmado:** conflito verificável é o único requisito inegociável. Fonte primária canônica e marca reconhecível são desejáveis, não obrigatórias.

Schema do objeto de decisão

Validado contra um par de conflito real (ver abaixo). Este é o menor schema que ainda detecta um conflito honesto:

```
{
  "empresa": string,
  "topic": string,      // taxonomia FIXA (ex: message-queue, database, api-style)
  "subject": string,    // a tecnologia em questão (ex: Kafka)
  "verdict": enum,      // adopted | rejected | kept
  "context": string,    // situação da empresa — CORAÇÃO do card
  "reason": string,     // porquê ancorado no contexto — CORAÇÃO do card
  "source_url": string,
```

```
"tags": string[] // livre - busca e descoberta de eixos novos
}
```

topic fixo + tags livres: o `topic` usa taxonomia fixa para que a detecção de conflito seja confiável (strings batem). As `tags` são livres para busca e para revelar, na prática, eixos recorrentes que mereçam virar topic canônico depois.

Regra de detecção de conflito

A regra ingênua `mesmo topic + mesmo subject + verdict oposto` está **incompleta**. Conflito sem contexto divergente é ruído, não sinal.

A regra honesta é:

- `mesmo topic + mesmo subject + verdict oposto` → marca como **conflito candidato**
- o card só vale se os dois `context` / `reason` **explicam a divergência** — ambas as empresas certas para seu próprio contexto

A primeira parte é um `WHERE` no SQL. A segunda é qualitativa e depende da qualidade da extração. Por isso a detecção de conflito é, no fundo, um problema de qualidade de extração do `reason`, não um problema de query.

Par de conflito real (validado à mão)

Eixo `message-queue` — o único confirmado ponta a ponta. Extraído manualmente de fontes reais, sem LLM, para servir de gabarito do que a saída do pipeline precisa parecer.

Dagster

REJECTED

topic: message-queue

subject: Kafka

context: startup pequena, time enxuto, logs de pipelines de dados em near-real-time

reason: custo operacional de adotar infra nova não se justifica; Postgres já estava no stack e resolveu

source: dagster.io/blog

vs

Disney+ Hotstar

ADOPTED

topic: message-queue

subject: Kafka

context: 50M usuários simultâneos, 5 bilhões de eventos em evento ao vivo

reason: alto throughput e baixa latência eram o requisito central; Kafka entrega isso

source: blog de engenharia do Hotstar

Este par é conflito **honesto** porque os dois `context` são diferentes e visíveis: ambas responderam à mesma pergunta ("Kafka vale a pena para fila de mensagens?") e chegaram a respostas opostas por razões que o leitor mapeia para o próprio caso. Tire o contexto e vira só "duas empresas fazendo coisas diferentes" — que não é conflito informativo.

Aprendizado do processo: o exemplo "Stripe saiu do Kafka / Discord adotou Kafka" usado no início do planejamento **não sobreviveu à verificação**. O caso Discord é sobre banco de dados (Cassandra → ScyllaDB), não Kafka. Isto reforça por que a verificação de fontes precede qualquer código: premissas carregadas de cabeça estavam factualmente erradas.

Nota sobre seleção de eixos

Nem todo eixo gera conflito com a mesma facilidade. O eixo `monolito vs microservices`, na busca inicial, rendeu apenas **consenso** (Segment e Istio ambos voltaram ao monolito) — não conflito. Isso revela um viés de publicação: posts de engenharia são escritos sobre decisões surpreendentes ou dolorosas, super-representando reversões. O corpus precisa ser montado buscando dissidência verificável, não cobertura temática.

Stack

Camada	Tecnologia	Onde pode trair
Pipeline de ingestão	Python + LLM (OpenAI/Gemini) com saída estruturada em JSON	extração vem incompleta ou inventada; só se descobre auditando à mão
Banco	PostgreSQL + pgvector	tentar detectar conflito por embedding — não fazer; conflito é lógica sobre campos
Embeddings	API de embedding (mesmo provedor do LLM)	dimensão do vetor precisa bater com a coluna do pgvector
Backend	FastAPI	Vercel não roda FastAPI persistente bem
Frontend	React	gastar tempo demais polindo UI e fugindo da ingestão
Infra	Docker (Postgres local)	não containerizar por reflexo se o host já tem Postgres gerenciado
Hospedagem	Frontend no Vercel; backend + Postgres no Railway/Render	JSON e embeddings versionados no repo — demo nunca depende de ingestão ao vivo

Escopo: fatia vertical real

A demo roda a engenharia inteira sobre um corpus pequeno, fixo e curado — não dados inventados. Ingestão com LLM roda de verdade, embeddings gerados de verdade, busca semântica funciona de verdade, só que sobre dezenas de documentos em vez de milhares. "Dados fechados" é decisão de escopo, não atalho. O corpus é ingerido uma vez e o JSON resultante é versionado no repositório.

Estado atual e próximos passos

Item	Estado
Conceito e diferencial definidos	concluído
Direção visual (light mode, roxo #534AB7, ref. Linear/Stripe)	concluído
Design da landing + app view	concluído
Schema do objeto de decisão	validado contra 1 par real
Regra de conflito (com cláusula de contexto)	definida
Par de conflito real extraído à mão	1 par (message-queue)
Teste de extração com LLM nos 2 documentos	próximo passo
Mais pares de conflito (outros eixos)	em aberto
Pipeline de ingestão completo	não iniciado
Backend FastAPI + pgvector	não iniciado
Frontend React	não iniciado

Próximo passo concreto

Escrever o prompt de extração e rodá-lo nos 2 documentos reais (Dagster e Hotstar), comparando a saída do LLM com a extração feita à mão. Se o LLM reproduzir os campos — principalmente um `reason` ancorado em contexto, não genérico — a hipótese central está de pé e dá para escalar para o resto do eixo. Se não reproduzir, o problema real foi descoberto com 2 documentos e uma tarde, em vez de 50 documentos e duas semanas.

Princípio que guia a ordem de construção: o primeiro código não é `docker-compose` nem o pipeline genérico. É o menor teste possível da coisa mais arriscada — a extração. Montar a infra primeiro dá a sensação de progresso sem o progresso: ambiente bonito, zero decisões extraídas.

Design (referência visual)

Landing page (topo) e app view (base). Light mode, minimalista, roxo #534AB7 como cor de marca, referenciando a linguagem visual de Linear e Stripe.

arbiter

como funciona

fontes

docs

entrar no app

decisões reais de engenharia

o que times como stripe, discord e shopify decidiram — e por quê

descreva seu problema de arquitetura com contexto. o arbiter busca ADRs e eng blogs reais e mostra onde times diferentes chegaram a conclusões opostas.

começar a consultar

ver no github

✓ fontes reais linkadas

✓ conflitos explícitos

✓ sem respostas genéricas

ex: sistema de notificações, 50k usuários, time de 3 devs, latência tolerável de 2s...

buscar decisões

kafka vs rabbitmq

monolito vs microsserviços

postgres vs mongodb

rest vs graphql

redis vs memcached

DIFERENCIAIS

não é mais um chatbot de arquitetura



conflitos explícitos

quando stripe foi pro lado A e discord foi pro lado B, você vê os dois lados com o contexto de cada decisão.



fontes sempre linkadas

cada card mostra a fonte original — ADR público no github ou eng blog da empresa. sem respostas sem procedência.



contexto importa

a resposta muda com base no tamanho do time, volume e tolerância a complexidade. não é conselho genérico.

fontes indexadas

Stripe
eng blog

Discord
eng blog

Shopify
eng blog

Uber
eng blog

Netflix
eng blog

GitHub ADRs
público

Figma
eng blog

Hacker News
threads

pronto para consultar decisões reais?

sem cadastro. sem paywall. só contexto e decisões que existiram de verdade.

abrir o arbiter

app principal ↓

arbiter

consultar

salvos fontes

sistema de notificações · 50k usuários · time de 3 devs · latência tolerável 2s

buscar decisões

kafka vs rabbitmq

monolito vs microsserviços

postgres vs mongodb

rest vs graphql

4 decisões encontradas

⚠ conflito detectado — stripe vs discord

Stripe 2022

conflito

Kafka → SQS

overhead operacional do Kafka era alto para o tamanho do time. SQS resolveu 90% dos casos com 10% da complexidade.

↗ ver fonte ↗ ver conflito

Discord 2023

conflito

RabbitMQ → Kafka

volume cresceu além do suportado. Kafka permitiu replay de eventos e auditoria sem perda de mensagens.

↗ ver fonte ↗ ver conflito

Shopify 2021

manteve RabbitMQ

complexidade de Kafka não justificada. investiram em sharding do

Uber 2020

adotou Kafka como padrão

escala de trilhões de mensagens/dia. custo operacional absorvido pelo

RabbitMQ ao invés de migração completa.

↗ ver fonte

tamanho do time de infra dedicado.

↗ ver fonte

⚡ conflito — stripe vs discord

Stripe · 2022

saiu do Kafka

time pequeno, overhead operacional alto, throughput não era gargalo. SQS resolveu com muito menos complexidade.

stripe.com/blog

VS

Discord · 2023

migrou para Kafka

volume e necessidade de replay tornaram RabbitMQ insustentável. Kafka foi o único com as garantias de durabilidade necessárias.

discord.com/blog

O painel de conflito (base da imagem) só aparece quando existe conflito real nos dados — precisa ser condicional na implementação.